

Avro Linux

Developer Specification & AI Coding-Agent Implementation Plan

A complete technical specification for building a polished Debian/Linux Avro Phonetic keyboard using Linux-native IBus integration and AI coding agents in VS Code.

Primary reference: sarim/ibus-avro

Secondary reference: mugli/Avro-Keyboard

Target: Debian-based Linux; native .deb packaging

Governing document: docs/DEVELOPER_SPEC.md

1. Project Overview

Build a modern Linux implementation of Avro Keyboard functionality, with Avro Phonetic as the primary Bengali input method. Do not mechanically port the old Windows GUI; use Linux-native input-method infrastructure.

- Reliable Bengali phonetic typing through IBus.
- GTK, Qt and Electron compatibility.
- Native preferences application.
- English/Bangla mode.
- Phonetic conversion, dictionaries, suggestions and autocorrect where supported.
- Reproducible Debian packaging.
- Automated tests and compatibility matrix.
- No telemetry or transmission of typed content.

1.1 Product principle

IBus is the Linux integration boundary. Do not replace it with global keyboard hooks or a custom keyboard daemon.

1.2 Definition of success

A fresh Debian user can install the package, enable Avro through normal input-source configuration, type Bengali in common applications, configure it through native settings, upgrade/remove it safely, and reproduce the build.

2. Goals and Non-Goals

2.1 Goals

- Avro-compatible phonetic conversion.
- Correct IBus preedit and commit behavior.
- Backspace, delete, enter, escape, arrows and editing.
- Punctuation, numbers, capitalization and word boundaries.
- Dictionary, suggestions, autocorrect and user dictionary where supported.
- Native GTK preferences.
- GSettings/dconf configuration.
- Desktop launcher, icons and AppStream metadata.
- Debian-native packaging and CI.
- Documentation and privacy-preserving diagnostics.

2.2 Non-goals

- Do not rewrite the phonetic algorithm without evidence.
- Do not port the Windows Delphi/Pascal GUI wholesale.
- Do not use global keyboard hooks.
- Do not create a fake Windows tray architecture.
- Do not add cloud typing or telemetry.
- Do not copy assets with uncertain licensing.

3. Upstream and Reference Projects

3.1 Primary Linux reference: sarim/ibus-avro

<https://github.com/sarim/ibus-avro>

Use this as the primary implementation/reference because it already provides Avro phonetic input through Linux IBus.

- Inspect the complete tree before architectural changes.
- Identify engine entry point and IBus registration.
- Identify GJS modules, phonetic library and dictionaries.
- Identify autocorrect, suggestions and preferences.
- Identify GSettings, IBus XML, desktop and AppStream metadata.
- Inspect tests, limitations and licenses.

3.2 Secondary Windows reference: mugli/Avro-Keyboard

<https://github.com/mugli/Avro-Keyboard>

Use this only as a behavior and feature-parity reference. Do not port it wholesale.

- Compare expected user-visible behavior.
- Identify features users may expect.
- Do not assume Windows implementation details belong on Linux.
- Check license/provenance before copying anything.

3.3 Debian reference

Inspect Debian's ibus-avro packaging for current dependencies, paths and package conventions. Prefer Debian-native facilities.

4. Proposed Architecture

```
avro-linux/  
■■■ src/  
■   ■■■ engine/  
■   ■■■ avro-core/  
■   ■   ■■■ phonetic/  
■   ■   ■■■ dictionary/  
■   ■   ■■■ autocorrect/  
■   ■   ■■■ suggestions/  
■   ■■■ preferences/  
■   ■■■ desktop/  
■   ■■■ common/  
■■■ data/  
■   ■■■ icons/  
■   ■■■ applications/  
■   ■■■ ibus/  
■   ■■■ gsettings/  
■   ■■■ metainfo/  
■■■ tests/  
■   ■■■ core/  
■   ■■■ engine/  
■   ■■■ integration/  
■   ■■■ packaging/  
■■■ docs/  
■■■ debian/  
■■■ scripts/  
■■■ Makefile  
■■■ README.md
```

4.1 Component responsibilities

- avro-core: deterministic phonetic conversion, dictionaries, autocorrect and suggestions.
- engine: IBus-facing preedit, commit, key handling and lifecycle.
- preferences: GTK-native configuration UI backed by supported configuration APIs.
- common: shared constants/configuration helpers.
- data: desktop, IBus, GSettings, icon and AppStream resources.
- tests: unit, regression, integration and packaging tests.

- debian: package metadata and installation rules.

4.2 Architecture rule

The engine must remain usable without the preferences application. A broken settings window must not prevent IBus from starting.

5. Technology Stack

- Engine: GJS/JavaScript following the Linux upstream implementation.
- Input framework: IBus.
- Configuration: GSettings/dconf.
- Preferences: native GTK.
- Packaging: Debian debhelper and standard metadata.
- Build: Makefile plus Debian build tools.
- CI: GitHub Actions.
- Testing: core regression, engine, integration and package tests.

5.1 Dependency discipline

Avoid large frameworks without a concrete requirement. Document every non-standard dependency.

6. Functional Requirements

6.1 Input modes

- Bangla/Avro and English modes.
- Normal IBus mode switching.
- English mode must not interfere with applications.

6.2 Phonetic conversion

- Avro-style Latin-to-Bengali conversion.
- Vowels, consonants, vowel signs, hasanta and conjuncts.
- Word boundaries.
- Punctuation and numbers.
- Consistent capitalization.

6.3 Editing

- Correct backspace/delete behavior.
- Arrow keys must not corrupt conversion.
- Enter commits correctly.
- Escape cancels preedit where applicable.
- Focus changes reset/commit according to IBus semantics.

6.4 Dictionary and suggestions

- Preserve upstream dictionary format where possible.
- User dictionary support.
- Suggestions must never block basic typing.
- Graceful failure if dictionary/autocorrect resources are unavailable.

7. Preferences Application

7.1 Main UI

Provide a native settings application discoverable from the desktop application menu.

- General
- Typing
- Dictionary & Autocorrect
- Shortcuts
- Diagnostics
- About

7.2 General

- Basic feature settings.
- Mode/typing preferences.
- Reset all settings.

7.3 Typing

- Preedit-related settings where supported.
- Suggestion behavior.
- Autocorrect behavior.
- Word-boundary settings if available.

7.4 Diagnostics

- Version and IBus status.
- Configuration locations.
- Copyable diagnostic summary.
- Never include raw typed text.

7.5 About

- Version.
- Licenses.
- Upstream attribution.
- Repository/project information.
- Third-party notices.

8. IBus Integration

- Register through standard IBus component/engine XML.
- Use normal IBus lifecycle.
- Correct engine creation/reset/destruction.
- Handle focus in/out.
- Implement preedit and commit events.
- Use surrounding text where appropriate.
- Apply settings changes safely.
- Never use global keyboard hooks.

8.1 Compatibility targets

- GTK applications
- Qt applications
- Electron applications
- Firefox
- Chromium/Chrome
- LibreOffice
- VS Code
- Terminal applications

9. Configuration and Data

- Use GSettings for user-facing configuration.
- Keep defaults in installed schema.
- Use standard user config/data locations.
- Keep system dictionaries/assets read-only.
- Provide reset-to-default.
- Document every persistent setting.

9.1 Privacy

- No telemetry.
- No cloud processing.
- No transmission of typed content.
- No raw keystroke logging.
- Diagnostics contain metadata/status only.

10. Desktop Integration

- Desktop entry for preferences.
- Proper application icons.
- AppStream metadata.
- IBus component metadata.
- Correct application categories.
- Normal Debian desktop menu integration.
- No custom tray process required for normal operation.

11. Debian Packaging

11.1 Package requirements

- Valid .deb.
- Explicit runtime dependencies.
- Debian-standard file locations.
- Correct GSettings schema handling.
- Correct IBus XML installation.

- Correct desktop/AppStream metadata.
- Clean removal.
- Clean upgrade.

11.2 Build workflow

```
dpkg-buildpackage -us -uc
# or
debuild -us -uc
```

Verify package contents with `dpkg-deb` and test installation in a clean environment.

12. Testing Strategy

12.1 Core regression corpus

- Common phonetic words.
- Vowel/consonant combinations.
- Conjuncts and hasanta.
- Ambiguous phonetic sequences.
- Punctuation/numbers.
- Capitalization.
- Compound words.
- Backspace/editing sequences.
- Autocorrect.
- Dictionary/suggestions.

12.2 Engine tests

- Engine starts.
- IBus registration.
- Preedit.
- Commit.
- Reset.
- Focus transitions.
- Restart.
- Configuration changes.

12.3 Application matrix

- GTK editor
- Firefox
- Chromium/Chrome
- LibreOffice Writer
- VS Code/Electron
- At least one Qt application
- Terminal

13. AI Coding-Agent Rules

You are the implementation agent for Avro Linux.

1. Read docs/DEVELOPER_SPEC.md before modifying code.
2. Treat sarim/ibus-avro as the primary Linux upstream/reference.
3. Treat mugli/Avro-KeyBoard as secondary Windows behavior reference only.
4. Do not rewrite the phonetic algorithm unless explicitly required.
5. Preserve MPL-2.0 copyright/license obligations.
6. Never copy an asset when its license/provenance is uncertain.
7. Inspect source before changing architecture.
8. Keep every phase buildable.
9. Run relevant tests after every functional change.
10. Never claim a test passed unless it was actually run.
11. Never claim compatibility without testing it.
12. Do not use global keyboard hooks.
13. Do not store/transmit typed user content.
14. Do not add telemetry.
15. Prefer Debian-native facilities.
16. Keep engine and UI loosely coupled.
17. Every user-visible feature needs documentation and, where practical, a regression test.
18. When uncertain, inspect authoritative source/docs rather than guessing.
19. Before release, build in a clean environment and test install, typing, upgrade and uninstall.

14. Development Phases

Phase 0 — Reconnaissance

Inspect both upstream repositories, Debian packaging, dependencies, licenses, engine entry points, dictionaries and settings. Produce upstream-analysis.md, feature-matrix.md and licensing-notes.md.

Phase 1 — Repository and Packaging Skeleton

Create project structure, Debian skeleton, Makefile, README, licensing/attribution and CI. Produce a minimal installable .deb without changing Avro behavior.

Phase 2 — Engine Integration

Integrate/refactor the Linux engine while preserving behavior. Verify IBus registration, preedit and commit.

Phase 3 — Regression Suite

Build a deterministic phonetic corpus and engine tests. Freeze expected behavior.

Phase 4 — Preferences UI

Implement GTK settings backed by GSettings, including reset and diagnostics.

Phase 5 — Desktop Integration

Finalize icons, desktop entry, AppStream and IBus resources.

Phase 6 — Packaging and CI

Build .deb artifacts in CI and verify install/upgrade/removal.

Phase 7 — Compatibility Matrix

Test GNOME, target Debian versions and common GTK/Qt/Electron applications.

Phase 8 — Release Hardening

Review licenses, reproducibility, documentation, upgrade paths, crashes and final package contents.

15. First AI-Agent Prompt — Phase 0

You are the reconnaissance agent for the Avro Linux project.

Read docs/DEVELOPER_SPEC.md first.

Your task is ONLY Phase 0.

1. Inspect the complete sarim/ibus-avro repository.
2. Identify the engine entry point, IBus registration, GJS modules, phonetic conversion, dictionaries, autocorrect, suggestions, GSettings, preferences UI, desktop metadata, IBus XML,

- build/install files, tests, licenses and copyright.
- 3. Inspect mugli/Avro-Keyboard for feature/behavior parity only.
- 4. Inspect current Debian ibus-avro packaging conventions.
- 5. Create:
 - docs/upstream-analysis.md
 - docs/feature-matrix.md
 - docs/licensing-notes.md
- 6. Propose the new repository structure.

Do not rewrite the phonetic algorithm.
 Do not build the new UI yet.
 Do not change user-visible behavior.

Stop after Phase 0 and report exact files inspected, important findings, licensing findings, proposed architecture, commands actually executed and tests actually run.

16. Second AI-Agent Prompt — Phase 1

Read docs/DEVELOPER_SPEC.md and the completed Phase 0 documents.

Implement Phase 1 only.

1. Create repository structure.
2. Create Debian packaging skeleton.
3. Create a minimal installable .deb.
4. Create Makefile targets:
 - make
 - make test
 - make package
 - make clean
5. Create GitHub Actions CI.
6. Create README.md.
7. Add MPL-2.0 and third-party attribution notices as required.
8. Do not alter Avro phonetic behavior.
9. Build the package.
10. Run tests.
11. Verify package contents with dpkg-deb.

Stop after Phase 1.

Report exact commands executed, files changed, package filename, package contents and test results. Never claim a test passed unless it was actually run.

17. Repository Structure and File Ownership

src/engine/	IBus-facing runtime
src/avro-core/	phonetic, dictionaries, autocorrect, suggestions
src/preferences/	GTK preferences application
data/ibus/	IBus XML/component metadata
data/gsettings/	GSettings schema
data/applications/	desktop entry
data/metainfo/	AppStream metadata
data/icons/	application icons
tests/core/	deterministic language tests
tests/engine/	engine behavior tests
tests/integration/	desktop/application tests
tests/packaging/	package lifecycle tests
debian/	Debian metadata
docs/	developer/user documentation

The exact filenames may change after Phase 0 if upstream constraints justify it. Any architectural change must be documented.

18. Licensing and Attribution

- sarim/ibus-avro uses MPL-2.0; preserve applicable license headers and notices.
- Inspect each dictionary, icon, font, UI asset and data file individually.
- Create THIRD-PARTY/NOTICE material where appropriate.
- Do not misrepresent derived code as independently authored.

- Keep source references and license texts.
- Ensure package metadata identifies the applicable license.

The final release must contain enough attribution for users and distributors to identify upstream projects and licenses.

19. Release Checklist / Definition of Done

- Clean checkout builds.
- Dependencies documented.
- Regression tests pass.
- Engine starts under IBus.
- Bengali typing works in tested GTK, Qt and Electron apps.
- Firefox/Chromium/LibreOffice/VS Code/terminal tested where supported.
- Preferences launches and saves settings.
- Reset works.
- Desktop entry/icon installed.
- AppStream metadata validates.
- IBus metadata installed.
- Package installs cleanly.
- Package upgrades cleanly.
- Package removes cleanly.
- No typed content is transmitted.
- No telemetry.
- Licenses/third-party notices complete.
- Clean-environment build tested.
- README includes install, enable, configure, troubleshoot and uninstall.
- Known limitations documented.
- Version numbers consistent.

19.1 Acceptance criterion

A .deb existing on disk is not completion. Completion means a normal user can install it and use Avro reliably through Linux's input-method stack.

20. Final Guidance to the AI Agent

Work incrementally. Understand the existing Linux implementation first, preserve the proven phonetic engine, then build the Linux-native product layer around it.

- Inspect before editing.
- Prefer small reviewable changes.
- Keep every phase buildable.
- Run tests after functional changes.
- Record compatibility failures.
- Use authoritative documentation when uncertain.
- Do not make architecture changes solely because an AI-generated implementation appears cleaner.
- Treat typing behavior as compatibility-sensitive.

- Keep engine and UI independent.
- Use Debian conventions.
- Avoid unnecessary services.
- Never log keystrokes.
- Never add telemetry.
- Perform a fresh-machine installation test before release.

Suggested first repository state

```
git init avro-linux
cd avro-linux
mkdir -p docs
# Save this specification as:
# docs/DEVELOPER_SPEC.md
# Then give the Phase 0 prompt to the coding agent.
```

Reference URLs

Linux upstream: <https://github.com/sarim/ibus-avro>

Windows reference: <https://github.com/mugli/Avro-Keyboard>

Debian packages: <https://packages.debian.org/>

End of specification.